
Algorithm 1 ExpHash Inset Operation

```
1: procedure INSERT_KV_ITEM
2:   Input: key, value   Output: SUCCESS/FAIL
3:   kv  $\leftarrow$  Alloc_KV_Item(key, value)  $\triangleright$  alloc and init KV item along with prehash key
4:   (segment, depth)  $\leftarrow$  Dereference_Directory_Entry(key)  $\triangleright$  query target segment & depth
5:   level  $\leftarrow$  1
6:   bucket  $\leftarrow$  Get_Candidate_Bucket(segment, key, level)  $\triangleright$  candidate bucket at level
7:   lowest_vacant_slot  $\leftarrow$  NULL
8:   Prefetch(bucket)
9:   Retrieve_or_Execute_Bucket_Grained_Split()  $\triangleright$  harvest memory stalls

10:  BUCKET PROBING:
11:  fingerprint  $\leftarrow$  Calculate_Fingerprint(kv  $\rightarrow$  prehash, depth, level)
12:  slot_content  $\leftarrow$  Generate_Slot(kv, depth, level)
13:  for each slot in bucket do
14:    if slot is END then break  $\triangleright$  all slots invalidated by split operations
15:    else if slot is INVALID then continue  $\triangleright$  this slot invalidated by split operations
16:    else if slot is VACANT then
17:      if lowest_vacant_slot = NULL then lowest_vacant_slot  $\leftarrow$  slot  $\triangleright$  record the lowest vacant slot
18:      break
19:    else if fingerprint = slot.fingerprint and is_Key_Matched(key, slot) then
20:      return FAIL  $\triangleright$  target key already exists

21:  if is_Open_Addresssing_For_Deferrable_Chunk_Allocation(bucket) then
22:    adjacent_bucket  $\leftarrow$  Get_Bucket_For_Open_Addresssing(bucket)
23:    if Probe_Adjacent_Bucket(adjacent_bucket, key) then return FAIL  $\triangleright$  target key exists in adjacent bucket

24:  next_bucket  $\leftarrow$  Get_Candidate_Bucket(segment, key, level + 1)
25:  if next_bucket = NULL then
26:    if lowest_vacant_slot = NULL then
27:      new_chunk_size  $\leftarrow$  Chunk.Size(level + 1)  $\triangleright$  next-level chunk size
28:      if not is_Prefaulted_Pool_Sufficient(new_chunk_size) then
29:        adjacent_bucket  $\leftarrow$  Get_Bucket_For_Open_Addresssing(bucket)
30:        if Try_Insert_Adjacent_Bucket(adjacent_bucket, slot_content) then return SUCCESS
31:      numa_id  $\leftarrow$  GET_NUMA_ID_OF_SEGMENT(segment)
32:      new_chunk  $\leftarrow$  Alloc_Chunk(new_chunk_size, numa_id)  $\triangleright$  NUMA-aware allocation
33:      Initialize_Chunk_With_KV_Item_Insert(new_chunk, kv, depth, level + 1)
34:      Submit_Split_Request_To_Scheduler(segment, depth)
35:      if Try_Attach_Chunk_To_Segment(segment, new_chunk) then return SUCCESS  $\triangleright$  commit submitted
split request
36:      next_bucket  $\leftarrow$  Get_Candidate_Bucket(segment, key, level + 1)  $\triangleright$  other thread attached chunk
37:    else
38:      if Try_Insert_Slot(lowest_vacant_slot, slot_content) then return SUCCESS  $\triangleright$  insert into vacant slot
39:      goto BUCKET_PROBING  $\triangleright$  Retry probing on concurrent conflict
40:  level  $\leftarrow$  level + 1
41:  bucket  $\leftarrow$  next_bucket
42:  goto BUCKET_PROBING  $\triangleright$  Probe the bucket in the next level
```
